

Day 18: コメント投稿を実装しよう

今日のゴール

タスクに対してコメントを投稿できる機能を実装します。コメント履歴を表示し、新しいコメントを追加できるようにします。

【スクリーンショット: コメントセクション】

なぜこれを作るのか？

タスクの進捗や課題を共有するコミュニケーション機能です。コメントは付箋のようなもの。ノートに付箋を貼って補足情報を書き足すように、タスクにコメントを追加することで、背景情報や議論の履歴を残せます。後から見返したときに「なぜこうなったか」が分かります。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	コメント一覧表示	15分
Step 2	コメント投稿フォーム	15分
Step 3	コメント投稿API呼び出し	10分
Step 4	リアルタイム更新	10分

合計時間: 約50分

Step 1: コメント一覧表示（15分）

実装:

```

// filepath: src/components/task/TaskComments.tsx
'use client';

import { Box, Typography, Avatar, Paper } from '@mui/material';
import { api } from '@trpc/react';

interface TaskCommentsProps {
  taskId: string;
}

export function TaskComments({ taskId }: TaskCommentsProps) {
  const { data: comments, isLoading } = api.comment.getByTaskId.query(
    taskId,
  );

  if (isLoading) {
    return <Typography>読み込み中...</Typography>;
  }

  return (
    <Box sx={{ mt: 3 }}>
      <Typography variant="h6" sx={{ mb: 2 }}>コメント</Typography>
      {comments?.map((comment) => (
        <Paper key={comment.id} sx={{ p: 2, mb: 2 }}>
          <Box sx={{ display: 'flex', alignItems: 'center' }}>
            <Avatar sx={{ width: 32, height: 32, mr: 1 }} src={
              comment.user.name?.[0]
            } />
            <Typography variant="body2" fontWeight="bold">
              {comment.user.name}
            </Typography>
            <Typography variant="caption" color="text.secondary">
              {new Date(comment.createdAt).toLocaleString()}
            </Typography>
          </Box>
          <Typography variant="body2">{comment.content}</Typography>
        </Paper>
      ))}
    </Box>
  );
}

```

```
        </Paper>
      )})
    </Box>
  );
}
```

確認ポイント: コメント一覧が表示される

Step 2: コメント投稿フォーム（15分）

実装:

```
// filepath: src/components/task/TaskComments.tsx
import { useState } from 'react';
import { TextField, Button } from '@mui/material';

export function TaskComments({ taskId }: TaskCommentsProps) {
  const [content, setContent] = useState('');

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    console.log('コメント投稿:', content);
    setContent('');
  };

  return (
    <Box sx={{ mt: 3 }}>
      <Typography variant="h6" sx={{ mb: 2 }}>コメント</Typography>

      {/* コメント一覧 */}
      {comments?.map((comment) => (
        <Paper key={comment.id} sx={{ p: 2, mb: 2 }}>
          {/* 既存のコメント表示 */}
        </Paper>
      ))}

      {/* 投稿フォーム */}
      <Paper sx={{ p: 2, mt: 2 }}>
        <form onSubmit={handleSubmit}>
          <TextField
            fullWidth
            multiline
            rows={3}
            placeholder="コメントを入力..."
            value={content}
            onChange={(e) => setContent(e.target.value)}
            sx={{ mb: 2 }}
          />
        </form>
      </Paper>
    </Box>
  );
}
```

```
        <Button
          type="submit"
          variant="contained"
          disabled={!content.trim()}
        >
          投稿
        </Button>
      </form>
    </Paper>
  </Box>
);
}
```

確認ポイント: コメント入力フォームが表示される

Step 3: コメント投稿API呼び出し (10分)

実装:

```
// filepath: src/components/task/TaskComments.tsx
export function TaskComments({ taskId }: TaskCommentsProps) {
  const [content, setContent] = useState('');

  const createCommentMutation = api.comment.create.useMutation({
    onSuccess: () => {
      setContent('');
    },
  });

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!content.trim()) return;

    createCommentMutation.mutate({
      taskId,
      content: content.trim(),
    });
  };

  return (
    // UI は同じ
    <Button
      type="submit"
      variant="contained"
      disabled={!content.trim() || createCommentMutation.isLoading}
    >
      {createCommentMutation.isLoading ? '投稿中...' : '投稿'}
    </Button>
  );
}
```

確認ポイント: コメントが投稿される

Step 4: リアルタイム更新 (10分)

実装:

```
// filepath: src/components/task/TaskComments.tsx
export function TaskComments({ taskId }: TaskCommentsProp) {
  const utils = api.useUtils();

  const { data: comments } = api.comment.getByTask.useQuery(taskId);

  const createCommentMutation = api.comment.create.useMutation({
    onSuccess: () => {
      setContent('');
      utils.comment.getByTask.invalidate({ taskId });
    },
  });

  return (
    // UI は同じ
  );
}
```

確認ポイント: 投稿後すぐに新しいコメントが表示される

学んだこと

- Avatar コンポーネント: ユーザーアイコン表示 (イニシャル表示)
- Paper コンポーネント: カード風の背景
- toLocaleString: 日時を日本語フォーマットで表示
- trim(): 前後の空白を削除してバリデーション
- invalidate(): tRPCキャッシュを無効化して再取得

今日のまとめ

- ☐ コメント一覧を表示できた
- ☐ コメント投稿フォームを実装できた
- ☐ コメント投稿APIを呼び出せた
- ☐ リアルタイムで更新されるようにした

⚠ つまづきポイント

問題	原因	解決策
コメントが空でも投稿できる	trim() でチェックしていない	!content.trim() で無効化
投稿後にリストが更新されない	キャッシュが残っている	invalidate() でキャッシュクリア
日時がUTC表示になる	toLocaleString のロケール未指定	'ja-JP' を引数に渡す

次回予告

Day 19では、コメントの編集・削除機能を実装します。